

A. RISC-V Instructions

A.1 RV32I

Instruction	Format	Description	Operation
LUI	U-type	Load Upper Immediate	$rd = \text{imm}$
AUIPC	U-type	Add Upper Immediate to PC	$rd = pc + \text{imm}$
JAL	J-type	Jump and Link	$rd = pc + 4; pc = pc + \text{imm}$
JALR	I-type	Jump and Link Register	$rd = pc + 4; pc = (rs1 + \text{imm}) \& 1$
BEQ	B-type	Branch if Equal	if $(rs1 == rs2)$ $pc = pc + \text{imm}$
BNE	B-type	Branch if Not Equal	if $(rs1 \neq rs2)$ $pc = pc + \text{imm}$
BLT	B-type	Branch if $<$	if $(rs1 < rs2)$ $pc = pc + \text{imm}$
BGE	B-type	Branch if $\geq$	if $(rs1 \geq rs2)$ $pc = pc + \text{imm}$
BLTU	B-type	Branch if $<$ Unsigned	if $(rs1 < rs2)$ $pc = pc + \text{imm}$
BGEU	B-type	Branch if $\geq$ Unsigned	if $(rs1 \geq rs2)$ $pc = pc + \text{imm}$
LB	I-type	Load Byte	$rd = \text{sign-extend}(M[rs1 + \text{imm}][7:0])$
LH	I-type	Load Halfword	$rd = \text{sign-extend}(M[rs1 + \text{imm}][15:0])$
LW	I-type	Load Word	$rd = M[rs1 + \text{imm}]$
LBU	I-type	Load Byte Unsigned	$rd = \text{zero-extend}(M[rs1 + \text{imm}][7:0])$
LHU	I-type	Load Halfword Unsigned	$rd = \text{zero-extend}(M[rs1 + \text{imm}][15:0])$
SB	S-type	Store Byte	$M[rs1 + \text{imm}] = rs2[7:0]$
SH	S-type	Store Halfword	$M[rs1 + \text{imm}] = rs2[15:0]$
SW	S-type	Store Word	$M[rs1 + \text{imm}] = rs2$

Instruction	Format	Description	Operation
ADDI	I-type	Add Immediate	$rd = rs1 + imm$
SLTI	I-type	Set < Immediate	$rd = (rs1 < imm)$
SLTIU	I-type	Set < Immediate Unsigned	$rd = (rs1 < imm)$
XORI	I-type	XOR Immediate	$rd = rs1 \wedge imm$
ORI	I-type	OR Immediate	$rd = rs1 \vee imm$
ANDI	I-type	AND Immediate	$rd = rs1 \& imm$
SLLI	R-type	Shift Left Logical Immediate	$rd = rs1 \ll shamt$
SRLI	R-type	Shift Right Logical Immediate	$rd = rs1 \gg shamt$
SRAI	R-type	Shift Right Arithmetic Immediate	$rd = rs1 \gg shamt$
ADD	R-type	Add	$rd = rs1 + rs2$
SUB	R-type	Subtract	$rd = rs1 - rs2$
SLL	R-type	Shift Left Logical	$rd = rs1 \ll rs2$
SLT	R-type	Set <	$rd = (rs1 < rs2)$
SLTU	R-type	Set < Unsigned	$rd = (rs1 < rs2)$
XOR	R-type	XOR	$rd = rs1 \wedge rs2$
SRL	R-type	Shift Right Logical	$rd = rs1 \gg rs2$
SRA	R-type	Shift Right Arithmetic	$rd = rs1 \gg rs2$
OR	R-type	OR	$rd = rs1 \vee rs2$
AND	R-type	AND	$rd = rs1 \& rs2$

A.2 Environment Calls

Name	Call Number (a7)	Description	Inputs	Outputs
PrintInt	1	Prints an integer	a0 = integer to print	N/A
PrintFloat	2	Prints a floating point number	fa0 = float to print	N/A
PrintDouble	3	Prints a double precision floating point number	fa0 = double to print	N/A
PrintString	4	Prints a null-terminated string to the console	a0 = the address of the string	N/A
ReadInt	5	Reads an int from input console	N/A	a0 = the int
ReadFloat	6	Reads a float from input console	N/A	fa0 = the float
ReadDouble	7	Reads a double from input console	N/A	fa0 = the double
ReadString	8	Reads a string from the console	a0 = address of input buffer a1 = maximum number of characters to read	N/A
Sbrk	9	Allocate heap memory	a0 = amount of memory in bytes	a0 = address to the allocated block
Exit	10	Exits the program with code 0	N/A	N/A
PrintChar	11	Prints an ascii character	a0 = character to print (only lowest byte is considered)	N/A
ReadChar	12	Reads a character from input console	N/A	a0 = the character
GetCWD	17	Writes the path of the current working directory into a buffer	a0 = the buffer to write into a1 = the length of the buffer	a0 = -1 if the path is longer than the buffer
Time	30	Get the current time (milliseconds since 1 January 1970)	N/A	a0 = low order 32 bits a1 = high order 32 bits

MidiOut	31	Outputs simulated MIDI tone to sound card (does not wait for sound to end).	See MIDI note below	N/A
Sleep	32	Set the current thread to sleep for a time (not precise)	a0 = time to sleep in milliseconds	N/A
MidiOutSync	33	Outputs simulated MIDI tone to sound card, then waits until the sound finishes playing.	See MIDI note below	N/A
PrintIntHex	34	Prints an integer (in hexadecimal format left-padded with zeroes)	a0 = integer to print	N/A
PrintIntBinary	35	Prints an integer (in binary format left-padded with zeroes)	a0 = integer to print	N/A
PrintIntUnsigned	36	Prints an integer (unsigned)	a0 = integer to print	N/A
RandSeed	40	Set seed for the underlying Java pseudorandom number generator	a0 = index of pseudorandom number generator a1 = the seed	N/A
RandInt	41	Get a random integer	a0 = index of pseudorandom number generator	a0 = random integer
RandIntRange	42	Get a random bounded integer	a0 = index of pseudorandom number generator a1 = upper bound for random number	a0 = uniformly select from [0, bound]
RandFloat	43	Get a random float	a0 = index of pseudorandom number generator	fa0 = uniformly randomly selected from from [0,1]
RandDouble	44	Get a random double from the range 0.0-1.0	a0 = index of pseudorandom number generator	fa0 = the next pseudorandom
ConfirmDialog	50	Service to display a message to user	a0 = address of null-terminated string that is the message to user	a0 = Yes (0), No (1), or Cancel(2)

InputDialogInt	51	N/A	N/A	N/A
InputDialogFloat	52	N/A	N/A	N/A
InputDialogDouble	53	N/A	N/A	N/A
InputDialogString	54	Service to display a message to a user and request a string input	a0 = address of null-terminated string that is the message to user a1 = address of input buffer a2 = maximum number of characters to read (including the terminating null)	a1 contains status value. 0: OK status. Buffer contains the input string.-2: Cancel was chosen. No change to buffer.-3: OK was chosen but no data had been input into field. No change to buffer.-4: length of the input string exceeded the specified maximum. Buffer contains the maximum allowable input string terminated with null.
MessageDialog	55	Service to display a message to user	a0 = address of null-terminated string that is the message to user a1 = the type of the message to the user, which is one of: 0: error message 1: information message 2: warning message 3: question message other: plain message	N/A
MessageDialogInt	56	Service to display a message followed by a int to user	a0 = address of null-terminated string that is the message to user a1 = the int to display	N/A
Close	57	Close a file	a0 = the file descriptor to close	N/A

MessageDialogDouble	58	Service to display message followed by a double	a0 = address of null-terminated string that is the message to user fa0 = the double	N/A
MessageDialogString	59	Service to display a message followed by a string to user	a0 = address of null-terminated string that is the message to user a1 = address of the second string to display	N/A
MessageDialogFloat	60	Service to display a message followed by a float to user	a0 = address of null-terminated string that is the message to user fa1 = the float to display	N/A
LSeek	62	Seek to a position in a file	a0 = the file descriptor a1 = the offset for the base a2 is the beginning of the file (0), the current position (1), or the end of the file (2)	a0 = the selected position from the beginning of the file or -1 is an error occurred
Read	63	Read from a file descriptor into a buffer	a0 = the file descriptor a1 = address of the buffer a2 = maximum length to read	a0 = the length read or -1 if error
Write	64	Write to a file descriptor from a buffer	a0 = the file descriptor a1 = the buffer address a2 = the length to write	a0 = the number of characters written
Exit2	93	Exits the program with a code	a0 = the number to exit with	N/A

Open	1024	Opens a file from a path Only supported flags (a1) are read-only (0), write-only (1) and write-append (9). write-only flag creates file if it does not exist, so it is technically write-create. write-append will start writing at end of existing file.	a0 = Null terminated string for the path a1 = flags	a0 = the file decriptor or -1 if an error occurred
------	------	---	---	--

A.3 Assembler Directives

Name	Description
.align	Align next data item on specified byte boundary (0=byte, 1=half, 2=word, 3=double)
.ascii	Store the string in the Data segment but do not add null terminator
.asciz	Store the string in the Data segment and add null terminator
.byte	Store the listed value(s) as 8 bit bytes
.data	Subsequent items stored in Data segment at next available address
.double	Store the listed value(s) as double precision floating point
.end_macro	End macro definition. See .macro
.eqv	Substitute second operand for first. First operand is symbol, second operand is expression (like #define)
.extern	Declare the listed label and byte length to be a global data field
.float	Store the listed value(s) as single precision floating point
.globl	Declare the listed label(s) as global to enable referencing from other files
.half	Store the listed value(s) as 16 bit halfwords on halfword boundary
.include	Insert the contents of the specified file. Put filename in quotes.
.macro	Begin macro definition. See .end_macro
.section	Allows specifying sections without .text or .data directives. Included for gcc comparability
.space	Reserve the next specified number of bytes in Data segment
.string	Alias for .asciz
.text	Subsequent items (instructions) stored in Text segment at next available address
.word	Store the listed value(s) as 32 bit words on word boundary